

Identity, Onboarding, and Biological Continuity: A FEN Schema Supplement for Phoros

Marcus A. Thomas

May 28, 2026

1 Purpose of This Supplement

This document supplements the FEN schema by showing how identity, onboarding, biological continuity, account recovery, institutional record linkage, and delegated authority are represented using the core FEN abstractions: **Fact**, **ProblemEpisode**, **EpisodeMembership**, and **Narrative**.

It does not introduce a parallel identity architecture. Identity events are time-indexed, provenance-bearing, evidence-grounded events. They have the same structure as measurements, prescriptions, encounters, coverage events, claims, and pre-authorization decisions.

The central claim is:

Identity is not a separate subsystem layered beside FEN. It is a high-assurance region of the FEN fact graph. A biologically continuous person is anchored by **SubjectId**; identity, continuity, institutional links, and authority are represented as provenance-bearing **FactPayload** variants; onboarding, recovery, delegation, and dispute resolution are episode-like workflows; and the current identity state is a derived narrative or materialized view over those facts.

This supplement extends the admissible **FactPayload** variants, adds a small number of supporting value types, and proposes an optional **EpisodeKind** enum to distinguish clinical, administrative, identity, access, recovery, and delegation workflows while preserving the meaning of **ProblemEpisode**.

2 The Identity Problem

The identity problem is the problem of determining who is acting, which biologically continuous person they correspond to, which clinical and institutional identities are linked to that person, and what authority they have to act.

This is a problem in the same sense FEN treats clinical management threads as problems. It is not a disease episode, but it is a persistent epistemic and practical problem. The system must solve it fallibly, preserve the evidence used to solve it, revise when better evidence arrives, and represent disagreement when evidence is incomplete.

The governing principle of this supplement follows directly:

The answer to the identity problem is never a primitive fact of the system. Identity-relevant inputs—credential assertions, government ID checks, liveness checks, biometric continuity, portal links, attestations, delegations—are *evidence*. No single one of them

is identity, continuity, or authority. The system accumulates and evaluates them; it does not store a mutable field asserting that an account “is” a patient.

This principle is stated once here and assumed throughout. The remaining sections do not repeat the caveat after each fact.

The identity problem has several separable components:

- **Account authentication:** who controls the app account or session?
- **Biological continuity:** is the acting person continuous with the person previously enrolled or verified?
- **Canonical clinical identity:** which biologically continuous person is the durable subject of the record?
- **Institutional identity linkage:** which provider records, payer records, portal accounts, or external patient identifiers are linked to that subject?
- **Authority:** what actions may this person perform, for themselves or for another person?
- **Recovery:** how is control re-established after loss of device, credential, or account access?
- **Post-onboarding re-verification:** when does the system require fresh evidence before a sensitive action?

Ordinary consumer software collapses these into login. Phoros must not. A person may sign in with Google or Apple and still lack a verified clinical identity. A person may have a verified clinical identity and still lack authority to act for a child, parent, or dependent adult. A person may have ordinary access to their own dashboard and still need step-up verification before exporting a record, delegating authority, linking a provider, changing recovery methods, or authorizing a data transaction.

3 Design Principle: Identity as a Region of the Fact Graph

Identity-relevant events have the structure of facts. A government ID verification occurs at a time, by a source system, with a result. A biometric enrollment reference is established at a time and points to a separate subsystem. A device binding is created at a time and may later be revoked. A portal link may become stale, contested, superseded, or entered in error. A caregiver delegation has a scope, may expire, and may be revoked. These are events and assertions with provenance, not properties of a user object.

The existing Fact structure already fits:

```
Fact {
    id:                FactId,
    subject_id:        SubjectId,
    occurred_at:        TemporalAnchor,
    code:               Option<CodedValue>,
    payload:            FactPayload,
    status:             FactStatus,
    provenance:         Provenance,
    external_refs:      Vec<ExternalRef>,
}
```

For identity, `subject_id` identifies the subject; `occurred_at` records when the event occurred; `payload` carries the type-specific content; `status` records active, superseded, or entered-in-error; `provenance` records who or what asserted the fact; and `external_refs` connects to provider records, payer records, identity vendors, portal accounts, or FHIR resources.

The source of truth is the graph of identity facts and their memberships, not a mutable identity table. The app may maintain a materialized view for speed, but that view is derived from active, non-retracted, non-superseded facts and memberships.

4 SubjectId and the Minimal Identity Anchor

`SubjectId` is the durable internal anchor for a biologically continuous person or other identity-bearing subject. It is the node to which identity facts attach, not the evidence for identity.

The minimal model may keep `SubjectId` as a primitive identifier. An explicit subject object, if used, should be sparse. It must not absorb provider MRNs, payer member IDs, portal links, caregiver relationships, or raw biometric data; those belong in facts.

```
Subject {
  id:                SubjectId,
  subject_kind:      SubjectKind,
  status:            SubjectStatus,
  created_at:        Timestamp,
  created_by:        Author,
}
```

```
pub enum SubjectKind {
  HumanPerson,
  Organization,
  Device,
  SystemAgent,
}
```

```
pub enum SubjectStatus {
  Active,
  Superseded {
    superseded_by: SubjectId,
    reason: SubjectSupersessionReason,
  },
  EnteredInError,
  Disputed,
}
```

A subject's kind is not the capacity in which the subject acts. A biologically continuous human person remains a `HumanPerson` whether acting as patient, caregiver, clinician, attorney, legal proxy, or witness. Those capacities are represented through authority facts, participation roles, and episode memberships, not through the subject's intrinsic kind.

5 Identity FactPayload Extensions

Identity follows the existing pattern: it extends the closed `FactPayload` enum. It requires no trait hierarchy or separate identity-event subsystem. Each variant is an event, assertion, or decision—not a mutable field on a user profile. A portal link is stored by creating a `ClinicalIdentityLinkEstablished` fact, not by updating a `portal_links` field. If the link is wrong, it is contested, superseded, or marked `EnteredInError`; if it changes, a new fact is created.

```
pub enum FactPayload {
    // Existing clinical, administrative, billing, and insurance variants...

    SubjectCreated {
        subject_kind: SubjectKind,
        stable_profile: StableIdentityProfile,
    },

    IdentityAttributeAsserted {
        attribute: IdentityAttribute,
        value: IdentityAttributeValue,
        confidence: MatchConfidence,
    },

    IdentityWitnessRecorded {
        witness_type: IdentityWitnessType,
        target_subject_id: SubjectId,
        assurance_level: AssuranceLevel,
        evidence_ref: Option<DocumentRef>,
        expires_at: Option<Timestamp>,
    },

    BiometricEnrollmentReferenceAdded {
        biometric_system: String,
        enrollment_ref: String,
        modality: BiometricModality,
    },

    BiometricContinuityCheck {
        biometric_system: String,
        enrollment_ref: String,
        result: ContinuityCheckResult,
        assurance_level: AssuranceLevel,
    },

    DeviceBindingEstablished {
        device_ref: DeviceRef,
        authenticator_type: AuthenticatorType,
        assurance_level: AssuranceLevel,
    },

    DeviceBindingRevoked {
        device_ref: DeviceRef,
        reason: Option<String>,
    },
}
```

```

CredentialAssertion {
    authenticator_type: AuthenticatorType,
    device_ref: Option<DeviceRef>,
    result: CredentialAssertionResult,
    assurance_level: AssuranceLevel,
},

ClinicalIdentityLinkEstablished {
    provider_org: OrganizationRef,
    external_patient_ref: ExternalRef,
    match_confidence: MatchConfidence,
},

ClinicalIdentityLinkContested {
    link_fact_id: FactId,
    reason: Option<String>,
},

PayerIdentityLinkEstablished {
    payer: String,
    member_ref: String,
    effective_period: Option<TimeInterval>,
},

PayerIdentityLinkContested {
    link_fact_id: FactId,
    reason: Option<String>,
},

AuthorityRelationshipEstablished {
    actor_subject_id: SubjectId,
    target_subject_id: SubjectId,
    authority_type: AuthorityType,
    scope: AuthorityScope,
    valid_period: Option<TimeInterval>,
    evidence_ref: Option<DocumentRef>,
},

AuthorityRelationshipRevoked {
    relationship_fact_id: FactId,
    reason: Option<String>,
},

AccountRecoveryEvent {
    method: RecoveryMethod,
    result: RecoveryResult,
    assurance_level: AssuranceLevel,
},

RiskEvaluationEvent {
    action: SensitiveAction,
    result: RiskEvaluationResult,
    required_assurance: AssuranceLevel,
}

```

```

    },

    AccessDecision {
        action: SensitiveAction,
        decision: AccessDecisionResult,
        relied_on_facts: Vec<FactId>,
        policy_refs: Vec<PolicyRef>,
    },
}

```

6 Value Types for Identity

The following value types support the identity variants. They are illustrative and should be refined as implementation requirements become clearer.

Three types must be kept distinct. **SubjectKind** (Section 4) says what kind of entity the subject is. **AuthorityType** says what authority one subject has with respect to another. **ParticipationRole** says what role a subject plays in a particular workflow, episode, fact, or decision. A single human person can be the subject of their own health data, a caregiver for a parent, a legal representative for a client, and a clinician acting for an organization. These are capacities, not intrinsic kinds.

Roles and profile.

```

pub enum ParticipationRole {
    RecordSubject,
    Actor,
    Delegate,
    Caregiver,
    ParentGuardian,
    LegalRepresentative,
    Clinician,
    OrganizationAgent,
    Witness,
}

```

Participation roles are contextual. They describe how a subject participates in a specific episode, relationship, witness event, access decision, or workflow; they do not classify the person once and for all.

```

pub struct StableIdentityProfile {
    pub legal_name: Option<String>,
    pub date_of_birth: Option<Date>,
    pub demographic_attributes: Vec<IdentityAttribute>,
}

pub enum IdentityAttribute {
    LegalName,
    DateOfBirth,
    Address,
    PhoneNumber,
    Email,
    SexAdministrative,
}

```

```

        Other(String),
    }

    pub enum IdentityAttributeValue {
        StringValue(String),
        DateValue(Date),
        CodedValue(CodedValue),
    }

```

Witnesses, assurance, and confidence.

```

pub enum IdentityWitnessType {
    GovernmentIdVerification,
    SelfieLivenessCheck,
    BiometricContinuityCheck,
    PatientPortalLoginProof,
    ProviderAttestation,
    InPersonClinicVerification,
    PayerVerification,
    InsuranceCardVerification,
    DemographicMatch,
    DeviceBoundPasskeyAssertion,
    RecoveryKeyPresentation,
    LegalDocument,
}

```

`AssuranceLevel` grades the strength of a witness or decision. `MatchConfidence` grades how strongly an asserted attribute or institutional link matches a known record; its `Ambiguous` and `Conflicting` variants signal that manual review or contestation may be required.

```

pub enum AssuranceLevel {
    Low,
    Medium,
    High,
    VeryHigh,
}

pub enum MatchConfidence {
    Low,
    Medium,
    High,
    Exact,
    Ambiguous,
    Conflicting,
}

pub enum BiometricModality {
    Face,
    Fingerprint,
    Voice,
    Palm,
    Other,
}

```

Result types. Four small enums record outcomes. `ContinuityCheckResult` and `CredentialAssertionResult` share a {success, failure, inconclusive} shape (`Passed/Failed/Inconclusive` and `Succeeded/Failed/Inconclusive` respectively). `RiskEvaluationResult` and `AccessDecisionResult` additionally distinguish step-up and manual-review outcomes.

```
pub enum ContinuityCheckResult { Passed, Failed, Inconclusive }
pub enum CredentialAssertionResult { Succeeded, Failed, Inconclusive }

pub enum RiskEvaluationResult {
    Passed,
    Failed,
    RequiresStepUp,
    RequiresManualReview,
}

pub enum AccessDecisionResult {
    Allowed,
    Denied,
    StepUpRequired,
    ManualReviewRequired,
}
```

Authenticators.

```
pub enum AuthenticatorType {
    Passkey,
    PlatformBiometric,
    HardwareSecurityKey,
    AppPushMfa,
    RecoveryKey,
    Password,
    Other(String),
}
```

Authority and authorized actions.

```
pub enum AuthorityType {
    SelfAuthority,
    CaregiverDelegation,
    ParentGuardian,
    LegalProxy,
    PowerOfAttorney,
    AttorneyClientRepresentative,
    EmergencyAccess,
    ProviderTreatmentAuthority,
    OrganizationAgentAuthority,
}

pub struct AuthorityScope {
```

```

    pub permitted_actions: Vec<AuthorizedAction>,
    pub constrained_by_policy: Vec<PolicyRef>,
}

pub enum AuthorizedAction {
    ViewRecord,
    UploadDocument,
    ShareRecord,
    ScheduleCare,
    ManageBilling,
    LinkProvider,
    ExportRecord,
    AuthorizeDataTransaction,
    DelegateAuthority,
    RevokeAuthority,
}

```

Recovery and sensitive actions.

```

pub enum RecoveryMethod {
    ExistingTrustedDevice,
    RecoveryKey,
    GovernmentIdAndLiveness,
    ProviderAttestation,
    PayerVerification,
    ManualReview,
}

pub enum RecoveryResult {
    Approved,
    Denied,
    PendingManualReview,
}

pub enum SensitiveAction {
    ViewRecord,
    ShareRecord,
    ExportCompleteRecord,
    LinkProvider,
    LinkPayer,
    ChangeRecoveryMethod,
    DelegateAuthority,
    RevokeAuthority,
    AuthorizeDataTransaction,
    EmergencyAccess,
}

```

Raw biometric data is not stored in ordinary FEN facts. FEN stores biometric enrollment references, continuity events, and their provenance. The biometric template, if retained, is held in a specialized hardened subsystem. The FEN fact records that a binding exists, which system holds it, the modality, when it was established, and the assurance consequences. The FEN security model already separates **Fact** from operational, plaintext-bearing representations and governs ma-

terialization through policy; raw biometric material is handled under that same separation rather than as an ordinary field.

7 ProblemEpisode as Identity Workflow

An onboarding episode is a **ProblemEpisode** in the Popperian sense: not a disease episode, but a persistent, fallible, corrigible, evidence-relative problem—determine whether this account, device, biological person, and clinical identity should be treated as bound for specified purposes.

Because FEN may contain clinical problems, administrative workflows, identity verification, delegation, data-sharing, and dispute-resolution workflows, an optional **EpisodeKind** field makes the kind explicit.

```
pub enum EpisodeKind {  
    ClinicalProblem,  
    AdministrativeWorkflow,  
    IdentityVerificationWorkflow,  
    AccountRecoveryWorkflow,  
    DelegationWorkflow,  
    AccessAuthorizationWorkflow,  
    DataSharingWorkflow,  
    DisputeResolutionWorkflow,  
}
```

```
ProblemEpisode {  
    id:                ProblemEpisodeId,  
    subject_id:        SubjectId,  
    episode_kind:      EpisodeKind,  
    label:              String,  
    problem_code:      Option<CodedValue>,  
    status:            EpisodeStatus,  
    onset:              Option<ApproximateDate>,  
    authored_by:        Author,  
    authored_at:        Timestamp,  
    notes:              Option<String>,  
}
```

`episode_kind` does not change the ontology of episodes. It records which kind of problem or workflow an episode represents.

8 EpisodeMembership for Identity Evidence

An onboarding episode is defined by its membership records. A government ID check, liveness check, device binding, portal link, payer match, demographic match, and provider attestation are distinct facts. Their relevance to the onboarding problem is represented by **EpisodeMembership**, not by embedding them in the episode. Fact-to-episode assignment is itself an authored, provenance-bearing, contestable claim.

```
pub enum FactRole {  
    // Existing roles:
```

```

    TriggeringSymptom,
    DiagnosticTest,
    Treatment,
    OutcomeMeasure,
    Monitoring,
    Complication,
    Referral,
    Administrative,
    InsuranceAction,

    // Identity and access roles:
    IdentityAnchor,
    IdentityWitness,
    ContinuityWitness,
    DeviceBinding,
    InstitutionalLink,
    AuthorityEvidence,
    RecoveryEvidence,
    RiskSignal,
    AccessDecisionEvidence,
    DisputeEvidence,

    Other,
}

```

The same fact can play different roles in different episodes. A biometric continuity check may be a **ContinuityWitness** in a login episode and **AccessDecisionEvidence** in a record-export episode. A provider attestation may be an **IdentityWitness** in onboarding and **DisputeEvidence** in a later identity-linking disagreement.

9 Narrative as Current Identity State

The current identity state is a derived interpretation over active identity facts and memberships, not a mutable truth field. A **Narrative** summarizes which subject is believed to be the biologically continuous person, which institutional records are linked, what assurance has been reached, what authority relationships exist, and which questions remain open. Examples:

- This account is bound to **SubjectId** 123 at high assurance.
- The user passed government ID and liveness checks, but no provider portal link is established.
- Two institutional records match this subject; manual review is required.
- The caregiver delegation is valid for scheduling and record viewing, but not for data transactions.
- A prior portal link is contested and must not be relied on for high-risk access.

Narratives may coexist. The system maintains an operational narrative for authorization. A patient may author their own narrative of a disputed link. A provider or compliance reviewer may author a separate narrative explaining why a match was accepted, rejected, or held.

10 Onboarding: A Worked Walkthrough

Onboarding is a sequence of facts grouped into one or more problem episodes. The walkthrough below uses concrete values.

Episode. The system creates a `SubjectId` and an identity verification episode.

```
ProblemEpisode {
  id: onboarding_episode_id,
  subject_id,
  episode_kind: EpisodeKind::IdentityVerificationWorkflow,
  label: "Initial patient identity binding".to_string(),
  problem_code: None,
  status: EpisodeStatus::Active,
  onset: Some(onboarding_start),
  authored_by: system_author,
  authored_at: timestamp,
  notes: None,
}
```

Account bootstrap and device binding. The user creates an account and binds a passkey-enabled device. Sign in with Apple or Google, if used, is a convenience authenticator or bootstrap mechanism only.

```
SubjectCreated {
  subject_kind: SubjectKind::HumanPerson,
  stable_profile: StableIdentityProfile { ... },
}

DeviceBindingEstablished {
  device_ref,
  authenticator_type: AuthenticatorType::Passkey,
  assurance_level: AssuranceLevel::Medium,
}
```

Biological continuity enrollment. The system creates a biometric enrollment reference. The raw template stays out of FEN.

```
BiometricEnrollmentReferenceAdded {
  biometric_system: "ReservoiraryBiometricVault".to_string(),
  enrollment_ref: "enrollment_abc123".to_string(),
  modality: BiometricModality::Face,
}
```

Identity witness collection. The system collects witnesses.

```
IdentityWitnessRecorded {
  witness_type: IdentityWitnessType::GovernmentIdVerification,
```

```

    target_subject_id: subject_id,
    assurance_level: AssuranceLevel::High,
    evidence_ref: Some(government_id_document),
    expires_at: None,
}

IdentityWitnessRecorded {
    witness_type: IdentityWitnessType::SelfieLivenessCheck,
    target_subject_id: subject_id,
    assurance_level: AssuranceLevel::High,
    evidence_ref: None,
    expires_at: Some(timestamp),
}

```

Institutional record linkage. With sufficient evidence, the system links the subject to provider and payer records. These links remain external and defeasible; they may be superseded, contested, or entered in error.

```

ClinicalIdentityLinkEstablished {
    provider_org,
    external_patient_ref,
    match_confidence: MatchConfidence::High,
}

PayerIdentityLinkEstablished {
    payer: "Example Health Plan".to_string(),
    member_ref: "...".to_string(),
    effective_period: Some(period),
}

```

Memberships. Each evidence-bearing fact is linked to the episode by an `EpisodeMembership`, which records why the fact is relevant and who asserted that relevance.

```

EpisodeMembership {
    id: membership_id,
    fact_id: government_id_fact,
    episode_id: onboarding_episode_id,
    role: FactRole::IdentityWitness,
    asserted_by: system_author,
    asserted_at: TemporalAnchor::Point(timestamp),
    status: MembershipStatus::Active,
}

```

The same pattern applies to device binding, biometric enrollment, liveness, provider linkage, payer linkage, and manual review.

Narrative. The system creates a current narrative summarizing the state. The narrative interprets the underlying facts; it is not the source of truth.

```

Narrative {
  id: narrative_id,
  episode_id: onboarding_episode_id,
  author: system_author,
  authored_at: timestamp,
  status: NarrativeStatus::Current,
  summary: "Account bound to the subject at high assurance. \
    Government ID, liveness, device binding, and provider \
    portal linkage are active.",
  sections: vec![...],
  decision_points: vec![...],
}

```

The patient is now identity-bound for ordinary use. Higher-risk actions still require fresh step-up evidence.

11 Post-Onboarding Login and Biological Continuity

After onboarding, ordinary login does not re-establish identity from scratch. It produces fresh credential and continuity evidence supporting the claim that the acting user remains continuous with the verified subject.

11.1 Ordinary Login

The user authenticates with a passkey, platform biometric, or device gesture. The system records a credential assertion. This is evidence of account and device continuity, not a new proof of legal or clinical identity.

```

CredentialAssertion {
  authenticator_type: AuthenticatorType::Passkey,
  device_ref: Some(device_ref),
  result: CredentialAssertionResult::Succeeded,
  assurance_level: AssuranceLevel::Medium,
}

```

11.2 Biological Continuity Check

When fresh continuity is required, the system records a continuity event, treated as a continuity witness.

```

BiometricContinuityCheck {
  biometric_system: "ReservoiraryBiometricVault".to_string(),
  enrollment_ref: "enrollment_abc123".to_string(),
  result: ContinuityCheckResult::Passed,
  assurance_level: AssuranceLevel::High,
}

```

11.3 Step-Up Verification

Some actions require more than ordinary login: exporting the complete record, delegating authority, linking a provider or payer, changing recovery methods, authorizing a data transaction, revoking authority, or initiating emergency access. For these, the system may require a fresh continuity check, a recent device-bound credential assertion, a server-side risk evaluation, and possibly an additional witness.

```
RiskEvaluationEvent {
  action: SensitiveAction::ExportCompleteRecord,
  result: RiskEvaluationResult::Passed,
  required_assurance: AssuranceLevel::High,
}

AccessDecision {
  action: SensitiveAction::ExportCompleteRecord,
  decision: AccessDecisionResult::Allowed,
  relied_on_facts: vec![
    recent_credential_assertion,
    recent_biometric_check,
    risk_evaluation_fact,
  ],
  policy_refs: vec![export_policy],
}
```

This is analogous to enterprise step-up authentication. The question is not whether the user can open the app, but whether the available witnesses satisfy the policy for a sensitive action.

12 Off-the-Shelf Identity Infrastructure and the FEN Boundary

Phoros should use off-the-shelf identity infrastructure for commodity security functions, but it must not outsource the ontology of identity. Commercial IAM systems provide authentication, MFA, passkeys, identity proofing, device registration, risk signals, and step-up flows. They do not define the Phoros identity model.

Identity vendor

Authentication, MFA, passkeys, device binding, identity proofing, liveness checks, risk signals, orchestration, and credential lifecycle

FEN identity layer

Subject identity, identity facts, biological continuity, institutional links, authority relationships, provenance, onboarding episodes, recovery episodes, dispute history, narratives, and policy-relevant state

The vendor answers questions such as: did the user authenticate? was MFA completed? was a passkey assertion accepted? was a government ID and liveness check completed? was the device trusted? did the risk engine classify this as low-risk? Phoros answers a different class: which **SubjectId** does this account correspond to? which biologically continuous person is represented? which provider, payer, and portal records are linked, and which facts support those links? which witnesses are stale, contested, superseded, or entered in error? who may act for whom, and with

what authority? which policy constraints are satisfied by which witnesses? what narrative summarizes the state? The vendor is a source of identity-relevant events. It is not the owner of the identity graph.

12.1 The Three-Layer View and the FEN Region

A companion architecture document describes the deployment topology as three layers: a platform credential layer, a Reservatory-controlled biological-continuity vault, and the FEN identity graph. This view is consistent with the present supplement, provided the three layers are read as *a topology of evidence and ownership, not three identity models*.

- The **platform credential layer**—passkeys, WebAuthn, Secure Enclave or Android Keystore, and any authentication vendor—produces credential and device witnesses (`CredentialAssertion`, `DeviceBindingEstablished`).
- The **biological-continuity vault**—a Reservatory-controlled, hardened subsystem—produces continuity witnesses (`BiometricContinuityCheck`) and holds biometric templates outside FEN. FEN stores only references and events.
- The **FEN identity graph** is the single identity model. It owns `SubjectId`, identity facts, authority relationships, access decisions, and narratives, and computes identity, authority, and access state.

There is one identity ontology, in FEN. The other two layers feed it evidence. “Three layers” describes where evidence is produced and where templates live, not three competing representations of identity.

Platform biometrics warrant a specific note. Apple’s `LocalAuthentication` framework returns only the result of an authentication operation, not raw biometric data,¹ and Android’s `BiometricPrompt` provides a system-mediated authentication flow rather than app access to raw biometric data.² Platform biometrics are therefore device-bound credential witnesses (`AuthenticatorType::PlatformBiometric`), not the biological-continuity substrate. The continuity substrate is the vault, which owns its own enrollment references and matching protocol.

12.2 Vendor Outputs as FEN Facts

A successful vendor event becomes a provenance-bearing **Fact**. The source system is recorded in **Provenance**; any vendor event, transaction, credential, or external user identifier is recorded through `ExternalRef`. For example, a government ID and liveness verification:

```
Fact {
  id:          fact_id,
  subject_id:  subject_id,
  occurred_at: TemporalAnchor::Point(timestamp),
  code:        None,
  payload:     FactPayload::IdentityWitnessRecorded {
    witness_type: IdentityWitnessType::GovernmentIdVerification,
    target_subject_id: subject_id,
    assurance_level: AssuranceLevel::High,
```

¹See <https://developer.apple.com/documentation/localauthentication/>.

²See <https://developer.android.com/identity/sign-in/biometric-auth>.

```

        evidence_ref: Some(document_ref),
        expires_at: None,
    },
    status:          FactStatus::Active,
    provenance:      Provenance {
        source_system: Some("IdentityProvider".to_string()),
        source_document: None,
        imported_at: timestamp,
        author: system_author,
    },
    external_refs:   vec![vendor_transaction_ref],
}

```

The FEN graph then determines how the fact is used. A government ID fact may support onboarding; a passkey assertion may support ordinary login; a continuity check and risk evaluation may support a record export; a failed or inconclusive event may trigger lockout, manual review, or a recovery episode.

12.3 Why the Vendor Should Not Own the Identity Model

A commercial provider represents users as accounts, credentials, authentication methods, sessions, directories, policies, and claims. The Phoros model must represent biologically continuous subjects, institutional links, authority relationships, evidence provenance, contested links, superseded witnesses, caregiver delegation, legal proxy authority, recovery history, and current narratives—domain facts, not authentication properties. Treating the vendor user object as the source of truth would collapse the identity model back into application login and obscure the distinctions between account control, biological continuity, clinical identity, institutional linkage, and authority.

Onboarding and step-up with a vendor reuse the patterns already shown. Each completed vendor step (account creation, passkey registration, device registration, government ID, liveness, phone or email verification, risk scoring, recovery enrollment) produces one or more FEN facts, linked by `EpisodeMembership` to the relevant workflow. A step-up flow produces an `AccessDecision` that records the facts relied on and the policy satisfied—not merely “the vendor allowed it.”

12.4 Portability

The FEN boundary preserves vendor portability. If Phoros changes providers, the old vendor’s events remain as historical facts with provenance and external references, and new vendor events are added as new facts. The materialized identity state depends on FEN facts, statuses, memberships, policies, and narratives, not on a vendor’s internal user identifier. This avoids lock-in at the most sensitive layer.

12.5 Design Rule

Use off-the-shelf identity infrastructure for commodity security functions. Do not use it as the source of truth for patient identity, biological continuity, clinical linkage, or authority.

In implementation terms: the app invokes vendor flows; the backend validates the signed results, claims, webhooks, or API responses; the backend translates relevant outcomes into FEN facts; `EpisodeMembership` links those facts to workflows; `Narrative` or a materialized view summarizes the state; and Phoros policies decide what actions are permitted.

13 Authority and Agency

Phoros is a patient-agency system, not only an identity system. Authority is modeled as a policy-constrained permission to act, supported by evidence-bearing facts—not as a simple role label. A human person may hold self-authority over their own record and, in other contexts, authority as a parent, caregiver, attorney, legal proxy, or clinician. These are authority facts and participation roles, not intrinsic kinds.

```
AuthorityRelationshipEstablished {
  actor_subject_id,
  target_subject_id,
  authority_type: AuthorityType::CaregiverDelegation,
  scope: AuthorityScope {
    permitted_actions: vec![
      AuthorizedAction::ViewRecord,
      AuthorizedAction::ScheduleCare,
    ],
    constrained_by_policy: vec![caregiver_policy],
  },
  valid_period: Some(period),
  evidence_ref: Some(delegation_document),
}

AuthorityRelationshipRevoked {
  relationship_fact_id,
  reason: Some("Patient revoked caregiver access".to_string()),
}
```

The current authority state is a derived view over active authority facts, revocation facts, validity periods, and policy constraints.

14 Recovery, Revocation, Contestation, Merge, and Split

Identity systems must represent failure: lost devices, phished credentials, wrong portal links, duplicate subjects, mistaken merges, changed legal authority, revoked caregiver permission, disputed provider matches. These are represented inside the fact graph, not outside the schema.

14.1 Account Recovery

A recovery process creates facts, grouped into an `AccountRecoveryWorkflow` episode with memberships linking the request, ID check, liveness check, manual review, and access decision.

```
AccountRecoveryEvent {
  method: RecoveryMethod::GovernmentIdAndLiveness,
  result: RecoveryResult::Approved,
  assurance_level: AssuranceLevel::High,
}
```

14.2 Device Revocation

A lost or compromised device produces a revocation fact. The prior binding fact remains in history; current-state views no longer treat it as an active trusted device.

```
DeviceBindingRevoked {  
    device_ref,  
    reason: Some("User reported device lost".to_string()),  
}
```

14.3 Witness Supersession

A stale or replaced witness is superseded via the existing `FactStatus` mechanism. Supersession does not imply the original was wrong; a newer or stronger version may now exist.

14.4 Contested Identity Links

A provider, payer, or portal link may be contested. A contested link remains in the audit trail while being excluded from high-risk access decisions.

```
ClinicalIdentityLinkContested {  
    link_fact_id,  
    reason: Some("Possible match to wrong patient record".to_string()),  
}
```

14.5 Merging Duplicate Subjects

If two `SubjectIds` refer to the same person, the system creates facts and episode relations documenting the merge. One subject may be marked **Superseded** by another while all prior facts remain auditable.

14.6 Splitting Incorrectly Merged Subjects

If one `SubjectId` was used for two distinct people, facts are reassigned or relinked through provenance-bearing correction facts and **EnteredInError** or supersession statuses. The split is represented as a dispute-resolution episode, not an invisible database operation.

15 Materialized Identity State

The application needs efficient answers: is the account bound to a subject? what is the current assurance level? which devices are trusted? which provider and payer records are linked? which authorities are active? are there unresolved disputes? what step-up evidence does this action require? A materialized view answers these.

```
pub struct MaterializedIdentityState {  
    pub subject_id: SubjectId,  
    pub assurance_level: AssuranceLevel,  
    pub active_devices: Vec<DeviceRef>,  
    pub active_clinical_links: Vec<ClinicalIdentityLinkView>,  
}
```

```

    pub active_payer_links: Vec<PayerIdentityLinkView>,
    pub active_authorities: Vec<AuthorityRelationshipView>,
    pub unresolved_disputes: Vec<FactId>,
    pub last_continuity_check: Option<FactId>,
}

pub struct ClinicalIdentityLinkView {
    pub provider_org: OrganizationRef,
    pub external_patient_ref: ExternalRef,
    pub match_confidence: MatchConfidence,
    pub source_fact_id: FactId,
}

pub struct PayerIdentityLinkView {
    pub payer: String,
    pub member_ref: String,
    pub effective_period: Option<TimeInterval>,
    pub source_fact_id: FactId,
}

pub struct AuthorityRelationshipView {
    pub actor_subject_id: SubjectId,
    pub target_subject_id: SubjectId,
    pub authority_type: AuthorityType,
    pub scope: AuthorityScope,
    pub valid_period: Option<TimeInterval>,
    pub source_fact_id: FactId,
}

```

This view is a derived projection over active, non-retracted, non-superseded facts and memberships. The source of truth remains the FEN graph.

16 Security and Policy Implications

Representing identity inside FEN has four security advantages.

First, identity facts carry the same provenance and policy controls as other facts. A provider can learn that identity was verified at high assurance without seeing the government ID document; a caregiver can prove delegated authority without accessing unrelated evidence; a risk engine can use continuity facts without raw biometric information.

Second, identity evidence can be minimized. Policies can rely on derived claims, assurance levels, and witness types without disclosing raw evidence. A policy may require a high-assurance government ID verification fact without revealing the ID image to every component checking the policy.

Third, the system distinguishes ordinary login from high-risk action authorization. A passkey assertion may suffice to view low-risk content. Exporting a record, delegating authority, or changing recovery methods may require fresh continuity, trusted-device confirmation, server-side risk evaluation, or manual review.

Fourth, the system preserves an audit trail. The question is not merely whether access was allowed, but which facts, policies, witnesses, and narratives supported the decision.

17 Worked Example: Post-Onboarding Step-Up

A patient attempts to export the complete longitudinal record. Ordinary login is insufficient; the policy requires fresh continuity evidence. The patient provides a passkey assertion and a fresh continuity check; the server performs a risk evaluation; the system records an access decision.

```
CredentialAssertion {
  authenticator_type: AuthenticatorType::Passkey,
  device_ref: Some(device_ref),
  result: CredentialAssertionResult::Succeeded,
  assurance_level: AssuranceLevel::Medium,
}

BiometricContinuityCheck {
  biometric_system: "ReservatoryBiometricVault".to_string(),
  enrollment_ref: "enrollment_abc123".to_string(),
  result: ContinuityCheckResult::Passed,
  assurance_level: AssuranceLevel::High,
}

RiskEvaluationEvent {
  action: SensitiveAction::ExportCompleteRecord,
  result: RiskEvaluationResult::Passed,
  required_assurance: AssuranceLevel::High,
}

AccessDecision {
  action: SensitiveAction::ExportCompleteRecord,
  decision: AccessDecisionResult::Allowed,
  relied_on_facts: vec![
    credential_assertion_fact,
    biometric_continuity_fact,
    risk_evaluation_fact,
  ],
  policy_refs: vec![complete_record_export_policy],
}
```

This may be grouped into an `AccessAuthorizationWorkflow` episode, or left unassigned if no durable workflow is needed.

18 Worked Example: Caregiver Delegation

A patient delegates limited access to a caregiver. The caregiver has their own `SubjectId` and completes whatever onboarding the intended scope requires. The patient authorizes the relationship; the caregiver is the **actor**, the patient the **target**.

```
AuthorityRelationshipEstablished {
  actor_subject_id: caregiver_subject_id,
  target_subject_id: patient_subject_id,
  authority_type: AuthorityType::CaregiverDelegation,
  scope: AuthorityScope {
    permitted_actions: vec![
```

```

        AuthorizedAction::ViewRecord,
        AuthorizedAction::ScheduleCare,
    ],
    constrained_by_policy: vec![caregiver_access_policy],
},
valid_period: Some(period),
evidence_ref: Some(patient_signed_delegation),
}

```

This is grouped into a `DelegationWorkflow` episode. The delegation narrative may state that the caregiver can view the record and schedule care but cannot export the complete record, authorize data transactions, change recovery methods, or delegate further.

When the caregiver acts, they authenticate as themselves—never with the patient’s credentials—and the system records an access decision relying on the caregiver’s own credential and continuity facts plus the active delegation fact.

```

AccessDecision {
    action: SensitiveAction::ViewRecord,
    decision: AccessDecisionResult::Allowed,
    relied_on_facts: vec![
        caregiver_credential_assertion_fact,
        caregiver_biometric_continuity_fact,
        caregiver_delegation_fact,
    ],
    policy_refs: vec![caregiver_access_policy],
}

```

Revocation creates a fact. The original delegation fact remains for audit; the materialized authority view no longer treats it as active.

```

AuthorityRelationshipRevoked {
    relationship_fact_id: caregiver_delegation_fact,
    reason: Some("Patient revoked delegation".to_string()),
}

```

19 Relationship to the Core FEN Schema

This supplement does not alter the core FEN ontology. It extends the closed set of `FactPayload` variants and optionally adds an `EpisodeKind` field. The mapping is:

FEN primitive	Identity-layer use
<code>SubjectId</code>	Durable anchor for a biologically continuous subject
<code>SubjectKind</code>	Intrinsic entity kind: human person, organization, device, or system agent
<code>AuthorityType</code>	Context-specific authority one subject has over another
<code>ParticipationRole</code>	Context-specific role played in a workflow, fact, or decision
<code>Fact</code>	Identity events, witnesses, links, decisions, and authority facts
<code>FactPayload</code>	Closed enum of identity-specific event types
<code>FactStatus</code>	Supersession, correction, error, and revocation semantics
<code>ExternalRef</code>	Provider, payer, portal, FHIR, and identity-vendor references
<code>ProblemEpisode</code>	Onboarding, recovery, delegation, access, and dispute workflows
<code>EpisodeMembership</code>	Provenance-bearing relevance of facts to workflows
<code>FactRole</code>	Role played by evidence inside an identity workflow
<code>Narrative</code>	Current interpretation of identity, assurance, and authority

Identity is not an attribute table. It is an evidence graph.

20 Summary

The identity layer rests on five principles.

First, `SubjectId` anchors the biologically continuous person but does not prove identity. `SubjectKind` records the intrinsic entity kind. Context-specific capacities—patient, caregiver, clinician, attorney, legal representative, witness—are represented through authority facts, participation roles, and episode memberships. The proof of identity is the graph of facts attached to the subject.

Second, identity, continuity, institutional linkage, and authority are `FactPayload` variants: time-indexed, provenance-bearing, status-bearing, externally referenceable.

Third, onboarding, recovery, delegation, access authorization, and dispute resolution are `ProblemEpisodes`—fallible, corrigible, evidence-relative problems solved over time.

Fourth, `EpisodeMembership` connects identity facts to workflows, keeping evidence relevance authored, contestable, and auditable.

Fifth, the current identity state is a narrative or materialized view over facts, not a mutable record. The source of truth remains the FEN graph.

21 The Biological-Continuity Substrate

This section specifies what sits behind "`ReservoirBiometricVault`" and the opaque `enrollment_ref` used in the identity facts. The vault is the subsystem that produces `BiometricContinuityCheck` facts. FEN never holds raw biometric material; it holds the reference and the signed result.

21.1 What the Substrate Verifies

The substrate verifies continuity, not identity. It never runs a gallery search (1:N) across subjects. The only question is 1:1: does the live presentation match *this subject's* enrollment reference. Each subject has an isolated, per-subject encrypted reference. There is no cross-subject database, no

watchlist, no linkage. This constraint removes most of the privacy and accuracy burden that makes biometrics hard, and it is the basis for defensibility under BIPA and GDPR Article 9.

21.2 Where the Template Lives and Where Matching Runs

The central fork is template location and match location.

- **On-device matching** (the Face ID pattern): template in the secure enclave, match local. Maximum privacy, no central store. But it collapses to “the device says it matched,” it is device-bound (lose the phone, lose the reference), and Reservatory cannot control the threshold or re-score. This is platform biometrics, not the vault.
- **Server-side matching**: encrypted template stored centrally, capture sent to the server, Reservatory matches. Reservatory controls the model and threshold and can re-score. The cost is a central template store and raw biometric in transit.
- **Hybrid** (the long-term target): template stored server-side but decrypted and matched only inside a confidential-computing enclave, or stored under a template-protection scheme (cancelable biometrics) so the at-rest form is non-invertible. Reservatory control without a plaintext honeypot.

The FEN security model already separates **Fact** from operational, plaintext-bearing representations and governs materialization by policy. Template encryption and isolation fall under that same separation.

21.3 Liveness and Capture-Path Integrity

Matching is not the hard part. 1:1 face matching with modern embeddings is effectively solved. The frontier is liveness and capture-path integrity: proving the presentation is a live human at a real sensor now, not a mask, a replay, or a deepfake injected into a virtual camera. Defeating injection requires three mechanisms working together:

1. a server-issued challenge nonce³, so the capture cannot be pre-recorded;
2. app and hardware attestation (Play Integrity, App Attest), to bind the capture to a genuine sensor path;
3. signal-level passive liveness on the frames.

³A nonce is a “number used once” — a random value generated for a single transaction and never reused.

In the continuity flow, it works like this. When the client starts a continuity check, FEN generates a fresh random nonce and sends it to the client. The client must include that exact nonce in the capture it sends through the vault, and the nonce gets bound into the signed assertion. FEN then checks that the returned nonce matches the one it just issued and hasn’t been seen before.

This does two things:

It blocks replay. An attacker who captured a valid past assertion can’t resubmit it, because its nonce is stale — FEN already retired that value and won’t accept it again.

It blocks pre-recording and injection. A deepfake or a recorded video can’t be prepared in advance, because the attacker can’t know the nonce until FEN issues it moments before. A valid response has to incorporate a value that didn’t exist a second ago. Combined with a short expiry window (the nonce is only valid for a few seconds), this forces the response to be generated live, against the current challenge.

The term is general, not specific to biometrics. Nonces appear in authentication protocols, API request signing, cryptographic handshakes (TLS), and blockchain proof-of-work. The common purpose is to guarantee freshness — to prove a message was produced now, in response to this specific challenge, rather than copied from before.

This is where commercial vendors hold real intellectual property. Plan to license this layer for a long time, even after matching and storage are internalized.

21.4 The Signed Continuity Assertion

The trust interface is identical across all phases. Whatever produces the result—a vendor, a licensed SDK in Reservatory infrastructure, or a Reservatory service—emits one signed assertion:

```
ContinuityAssertion {
    enrollment_ref,
    challenge_nonce,
    timestamp,
    result,           // Passed | Failed | Inconclusive
    derived_assurance, // maps to AssuranceLevel
    modality,
    model_version,
    pad_result,       // presentation-attack / liveness result
}
```

The assertion is signed with the vault’s key. FEN verifies the signature, checks the nonce against what it issued, and writes a **BiometricContinuityCheck** fact. The vault is its own trust domain with its own key. While that interface is stable, the substrate can be replaced without changing FEN or any policy—the same portability argument the supplement makes for authentication vendors, applied one layer down.

21.5 Phased Build

Phase 1. Buy the entire substrate from a liveness/match vendor. `enrollment_ref` points at the vendor enrollment. Reservatory owns only nonce issuance, attestation verification, challenge policy, assurance mapping, and FEN translation.

Phase 2. Move capture orchestration and template storage in-house, in a Reservatory `enrollment_ref` namespace, while licensing the PAD and matching SDK and running it in Reservatory infrastructure.

Phase 3. Add confidential-computing matching, a template-protection scheme, multi-modal support, formal PAD evaluation (ISO/IEC 30107-3), and FAR/FRR monitoring with threshold governance.

Thresholds and assurance mappings are policy, not constants. They live in the same policy layer FEN already references and can be tuned per action without redeployment.

21.6 Consent, Retention, and Failure as Facts

Two properties follow from representing this in the fact graph. Consent and retention become facts: a consent fact at enrollment, a retention or destruction schedule, and revocation or re-enrollment as facts. This covers most of what BIPA and GDPR require. Failure handling—false rejects, lockouts, fallback to other witnesses—becomes ordinary episodes and access decisions, not special-case code. A failed liveness check, an inconclusive match, or a stale nonce each becomes its own fact and can open a lockout, manual-review, or recovery episode.

21.7 Architecture

Figure 1 shows the capture-to-FEN path across three trust domains.

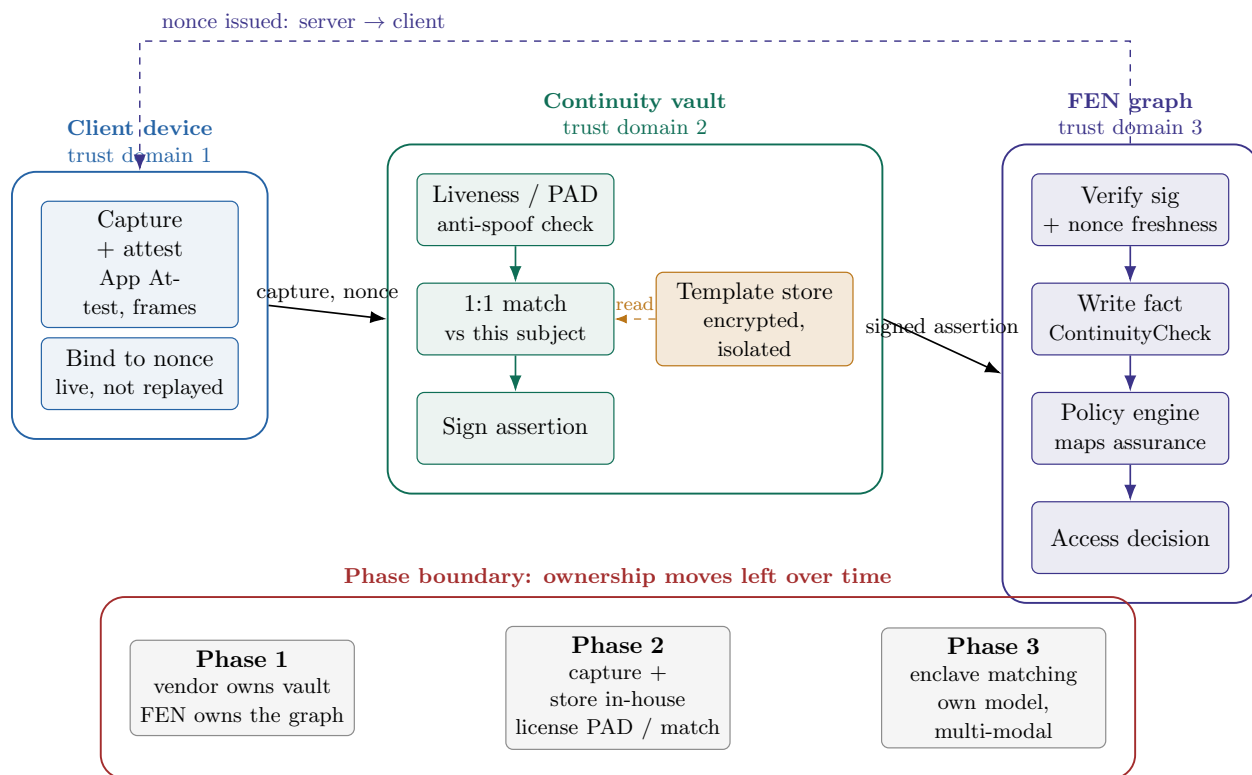


Figure 1: Biological-continuity substrate: the capture-to-FEN path across three trust domains. The client captures under a server-issued nonce and attestation; the vault runs liveness and 1:1 matching and emits a signed assertion; FEN verifies the signature and nonce, writes a `BiometricContinuityCheck` fact, and the policy engine produces an access decision. The template is read by the matcher and never leaves the vault.

Three commitments are encoded in the arrows.

First, the nonce arrow runs backward: FEN issues it, and the client must echo it through the vault. This is the defense against injection and replay. A deepfake fed to a virtual camera can produce a convincing face, but not one that incorporates a random challenge issued one second ago, unless the live capture path is compromised—which attestation exists to make hard. The nonce also makes the assertion non-replayable: FEN rejects any assertion whose nonce it did not just issue or has already seen.

Second, nothing flows out of the template store. The template is read into the matcher beside it and never leaves the vault, never crosses into FEN. A FEN breach therefore yields enrollment references and pass/fail results, not faces.

Third, the single arrow from *sign assertion* to *verify signature* is the portability seam. Everything left of that arrow can be a vendor, a licensed SDK, or a Reservatory enclave service without FEN changing, because FEN consumes only a signed assertion of known shape. The phase row is that seam sliding left: the boundary between bought and owned moves; the interface does not.

The diagram omits failure paths. A failed or inconclusive result does not return a bare “denied”—it becomes a fact and can open an episode, keeping failure auditable.